



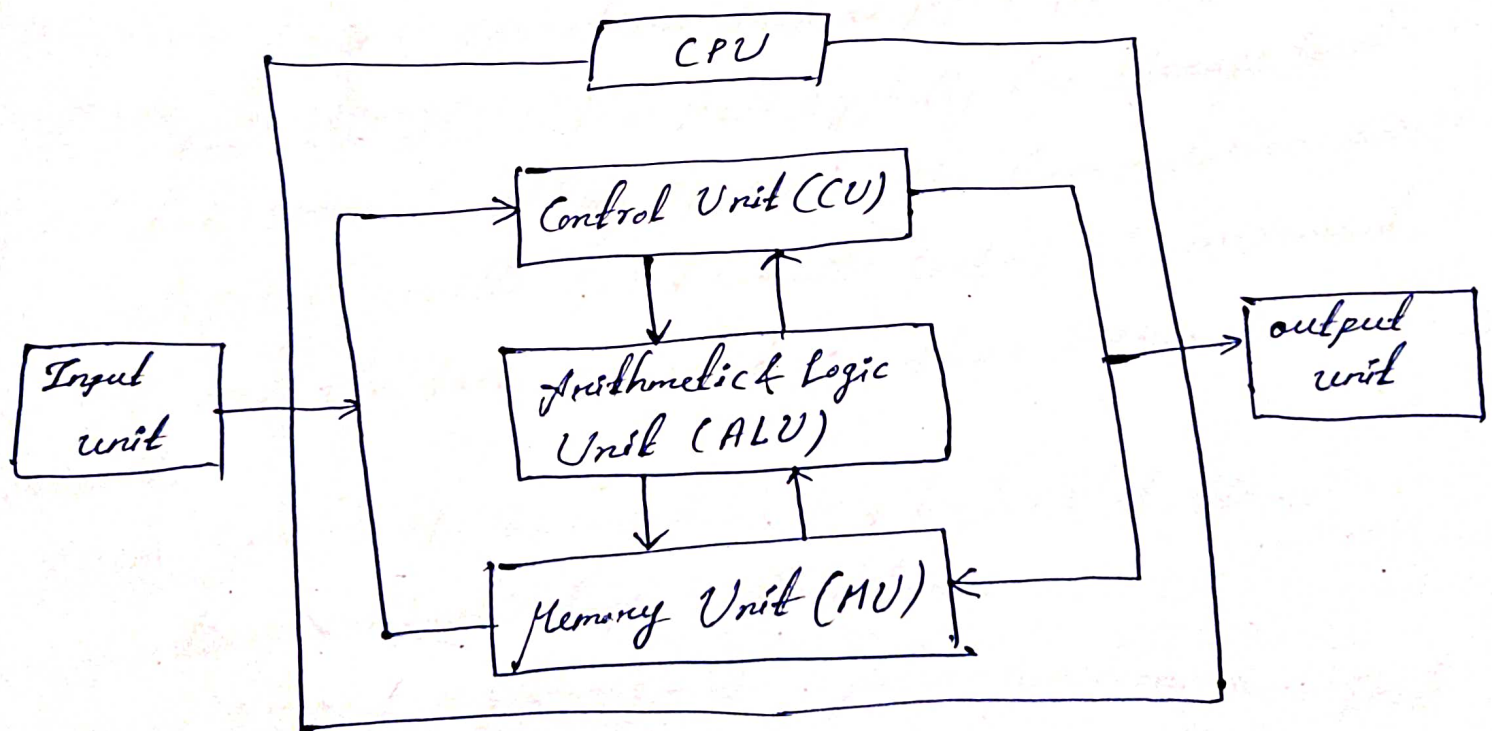
Topic:

Objective:

Outcomes:

Q.1 Write a short note on the design of the arithmetic unit. Explain the process of Arithmetic addition and subtraction of data in Computer ALU.

→ Arithmetic logic unit (ALU) is a circuit board embedded within a computer's central processing unit (CPU), which performs mathematical and logical operations using gateways made of electrical transistors that can convey signals in 0s & 1s.





Topic :

Objective :

Outcomes :

- Arithmetic unit :→ The arithmetic unit performs all numerical operations in a Computer, like addition and subtraction. It also performs other basic operations, such as the increment operation. At its most fundamental level, the ALU component combines two binary digits.
- Logic unit :→ The logic unit is the second half of the ALU. Instead of mathematical operations, the logic unit performs logical operations such as AND, OR, and XOR.
- Built-in storage :→ The ALU typically has storage locations for input operands, adding operands, the cumulative result, and shifting results. Gated circuits control the movement of bits and the tasks performed on them within the ALU's subunits.
- Accumulator :→ the accumulator is a buffer that stores intermediary arithmetic logic unit outcomes
- General purpose register :→ A register functions as a fast memory for receiving, storing and transmitting information.



Topic:

Objective:

Outcomes:

and instructions, which the ALU will require immediately.
Additions → Adds A and B with Carry-in or Carry-out sum at Y.
Subtractions → Subtracts B from A or vice versa with the difference at Y and Carry-in or Carry-out.

Q. 2 Explain the Sign-magnitude, 1's and 2's Complement. Also, define its range. Describe signed Addition and subtraction in detail.

→ In 1's Complement representation: → the representation of the positive number is same as the negative number, but the representation of the negative number is different.

for example: → if we want to represent -34 in 8-bit 1's Complement form, then first write the positive number (+34). And invert all 1s in that number by 0s and 0s by 1s in that number. The corresponding inverted number represents the -34 in 1's Complement form. It is also called 1's Complement of the number +34.

To represent -34 in 1's Complement form

$$+34 = 00100010$$

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

$$-34 = 11011101 \text{ (1's Complement of +34)}$$

Topic:

Objective:

Outcomes:

Here is another example which shows how to represent -60 in 8-bit 1's Complement form.

To represent -60 in 1's Complement form

$$+60 = 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0$$

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

$$-60 = 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \quad (1's \text{ Complement of } +60)$$

Using n -bits, the range of numbers that can be represented in 1's Complement form is from $-(2^{n-1}-1)$ to $(2^{n-1}-1)$.
For example, using 4-bits, it is possible to represent integers from -7 to $+7$ in a 1's Complement form representation. Similar to sign-magnitude form, there are two different representations of 0 in 1's Complement form representation.

$$+0 = 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0$$

$$-0 = 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1$$

2's Complement Representation: →

In 2's Complement representation also, the representation of the positive number is same as 1's Complement and sign magnitude form.



Topic:

Objective:

Outcomes:

$$+60 = 00111100 \text{ (sign magnitude Representation)}$$

$$+60 = 00111100 \text{ (1's Complement)}$$

$$+60 = 00111100 \text{ (2's Complement)}$$

but the representation of the negative number is different.
for example, if we want to represent -34 in 2's Complement
from then for example. Write 2's Complement of 34. Whose
1's Complement binary representation is given by 1001. This is
done by replacing the 0s with 1s and vice versa, i.e., 0110.
Now adding 1 to the inversion to obtain 2's Complement, i.e.
0111. Thus the 2's Complement of 34 is 0111.

Addition and Subtraction of Signed Magnitude Numbers:

Operations	Addition of Magnitudes	Subtraction of Magnitudes		
		$P > Q$	$P < Q$	$P = Q$
$(+P) + (+Q)$	$+(P+Q)$	$+(P-Q)$	$-(Q-P)$	$+(P-Q)$
$(+P) + (-Q)$		$-(P-Q)$	$+(Q-P)$	$+(P-Q)$
$(-P) + (+Q)$	$-(P+Q)$	$+(P-Q)$	$-(Q-P)$	$+(P-Q)$
$(-P) + (-Q)$				
$(+P) - (+Q)$	$+(P+Q)$			
$(+P) - (-Q)$				



Topic :

Objective :

Outcomes :

$(-P) - (+Q)$	$-(P+Q)$			
$(-P) - (-Q)$		$-(P-Q)$	$+(Q-P)$	$+(P-Q)$

addition algorithm :- $\rightarrow$

- ① When the signs of P & Q are equal, add the two magnitudes and Connect the sign of P to the output.
- ② When the signs of P and Q are different, Compare the magnitudes and subtract the smaller number from the greater number.
- ③ The signs of the output have to be equal as P in case $P > Q$ or the complement of the sign of P in case $P < Q$.
- ④ When the two magnitudes are equal, subtract Q from P and modify the sign of the output to positive.

subtraction algorithm :- $\rightarrow$

- ① When the signs of P and Q are different, add the two magnitudes and Connect the signs of P to the output.
- ② When the signs of P and Q are the same, Compare the

Topic:

Objective:

Outcomes:

magnitudes. The signs of the output have to be equal as P in case $P > Q$ or the complement of the sign of P in case $P < Q$.

(3) When the two magnitudes are equal, subtract Q from P and modify the sign of the output to positive.

Q.3 Explain how addition and subtraction are performed in fixed-point numbers.

→ Whenever you add two fixed-point numbers, you may need a carry bit to correctly represent the result. For this reason, when adding two B -bit numbers, the resulting value has an extra bit compared to the two operands used.

$$a = f_2(0.234375, 0, 4, 6);$$

$$c = a + a$$

$$c =$$

$$0.4688$$

DataTypeMode: Fixed-point: binary point scaling
Signedness: Unsigned
WordLength: 5

Objective :

Outcomes :

Fractionlength : 6

a. bin

ans = 1111

C. bin

ans = 11110

If you add or subtract two numbers with different precision, the radix point first needs to be aligned to perform the operation. The result is that there is a difference of more than one bit between the result of the operation and the operands.

$a = fi(pi, 1, 16, 13);$

$b = fi(0.1, 1, 12, 14);$

$c = a + b$

$c = 3.2416$

DataTypeMode : Fixed-point : binary point scaling

Signedness : Signed

Wordlength : 18

Fractionlength : 14



Topic:

Objective:

Outcomes:

equivalent to 8.25 in decimal.

Example $\rightarrow$ Floating point subtraction

• Representation: The input Operands are represented as

$a = -9 = 1-1010-00100000000$ and $b = 3.9375 = 0-1000-11111000-000$

• Sign extraction: As sign of a is negative and a is greater thus $s = 1$

• Exponent subtraction $\rightarrow E_a = 1010$ and $E_b = 1000$. Thus result of the subtraction is $E = 0010$.

• Shifting of mantissa of lesser number $\rightarrow$ The mantissa $M_b = 1-11111000000$ is shifted by 2 bit right and the result is $M_b = 0-00111110000$

• Result of the mantissa subtraction is 010100010000 . This leading Zero indicates that the result is lesser than 1.

• The output of the adder is left shifted by 1 bit as there is one leading Zero and the exponent value is decremented by 1-bit to get the correct results. The new mantissa value is now 01000100000 Choosing the last 11-bits from the LSB and exponent is 1001



Topic:

Objective:

Outcomes:

Q. 4. Draw and explain flow charts for addition and subtraction of floating-point numbers.

→ Example: Floating point Addition

• Representation: The input operands are represented as

$a = 4.5 = 0.1001001000000000$ and $b = 3.75 = 0.1000111000000000$

• Sign extraction: As both the numbers are positive then sign of the output will be positive. Thus $S = 0$.

• Exponent subtraction: $E_a = 1001$ and $E_b = 1000$. Thus

result of the subtraction is $E = 0001$.

• Shifting of mantissa of lesser number: → The mantissa $M_b = 1.111000000000$ is shifted by 1 bit right and the result

is $M_b = 0.111100000000$

• Result of the mantissa addition is 00001000000000 and generates a carry. This means the result is greater than 1.

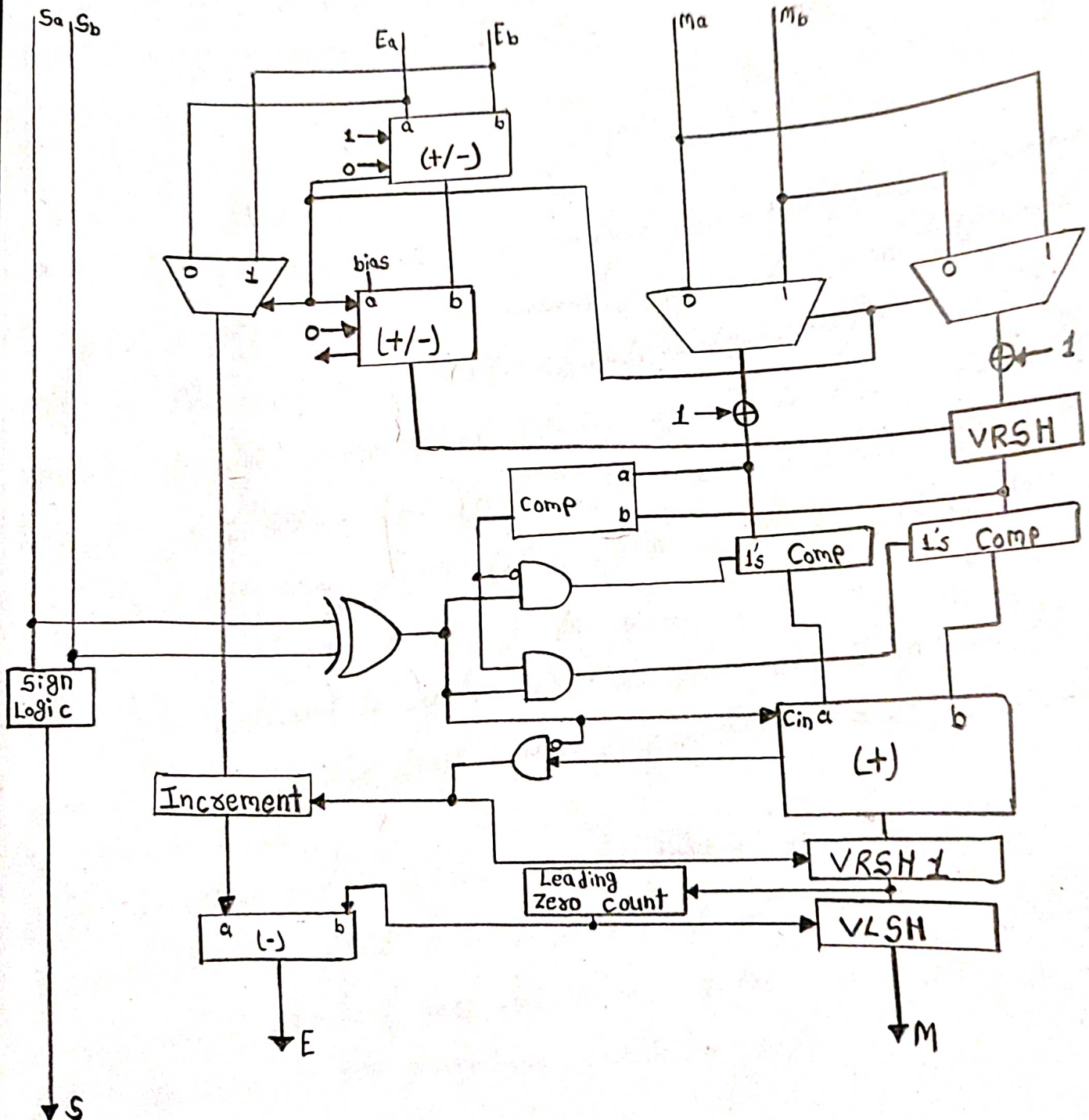
• The output of the adder is right shifted and the exponent value is incremented to get the correct results. The new mantissa value is now 000010000000 choosing the last 11-bits from the LSB and exponent is 1010 .

• The final result is 0.101000001000000000 which is

Topic:

Objective:

Outcomes:





Topic:

Objective:

Outcomes:

- The final result is $1-1001-01000100000$ which is equivalent to -5.0625 in decimal.

Q. 5 Write Booths Algorithm. Also, explain the floating-point Arithmetic Operation.

→ Booth algorithm gives a procedure for multiplying binary integers in signed 2's Complement representation in efficient way, i.e., less number of additions/subtractions required. It operates on the fact that strings of 0's in the multiplier require no addition but just shifting and a string of 1's in the multiplier from bit weight 2^k to weight 2^m can be treated as $2^{(k+1)}$ to 2^m .

- (1) The multiplicand is subtracted from the partial product upon encountering the first least significant 1 in a string of 1's in the multiplier.
- (2) The multiplicand is added to the partial product upon encountering the first 0 in a string of 0's in the multiplier.
- (3) The partial product does not change when the multiplier bit is identical to the previous multiplier bit.

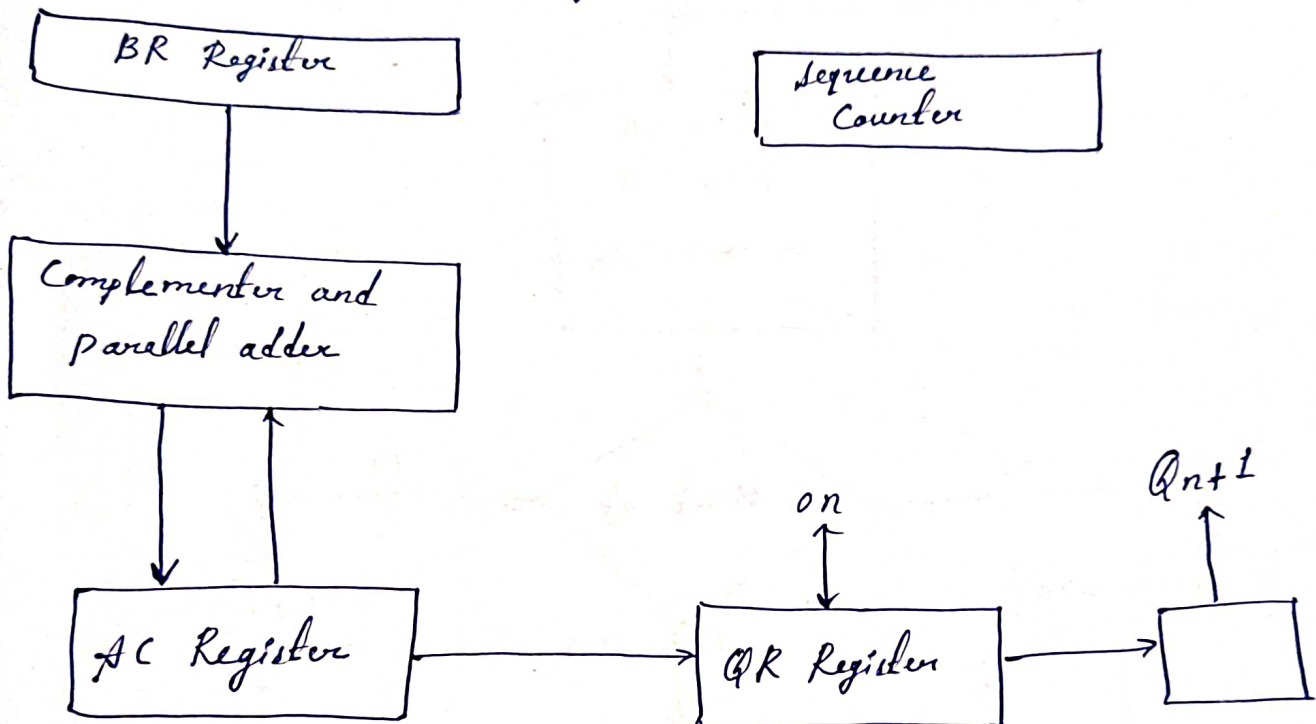
Topic :

Objective :

Outcomes :

• Hardware Implementation of Booth's Algorithm:

The hardware implementation of the booth algorithm requires the register configuration shown in the figure below.



Booth's Algorithm Flowchart :->

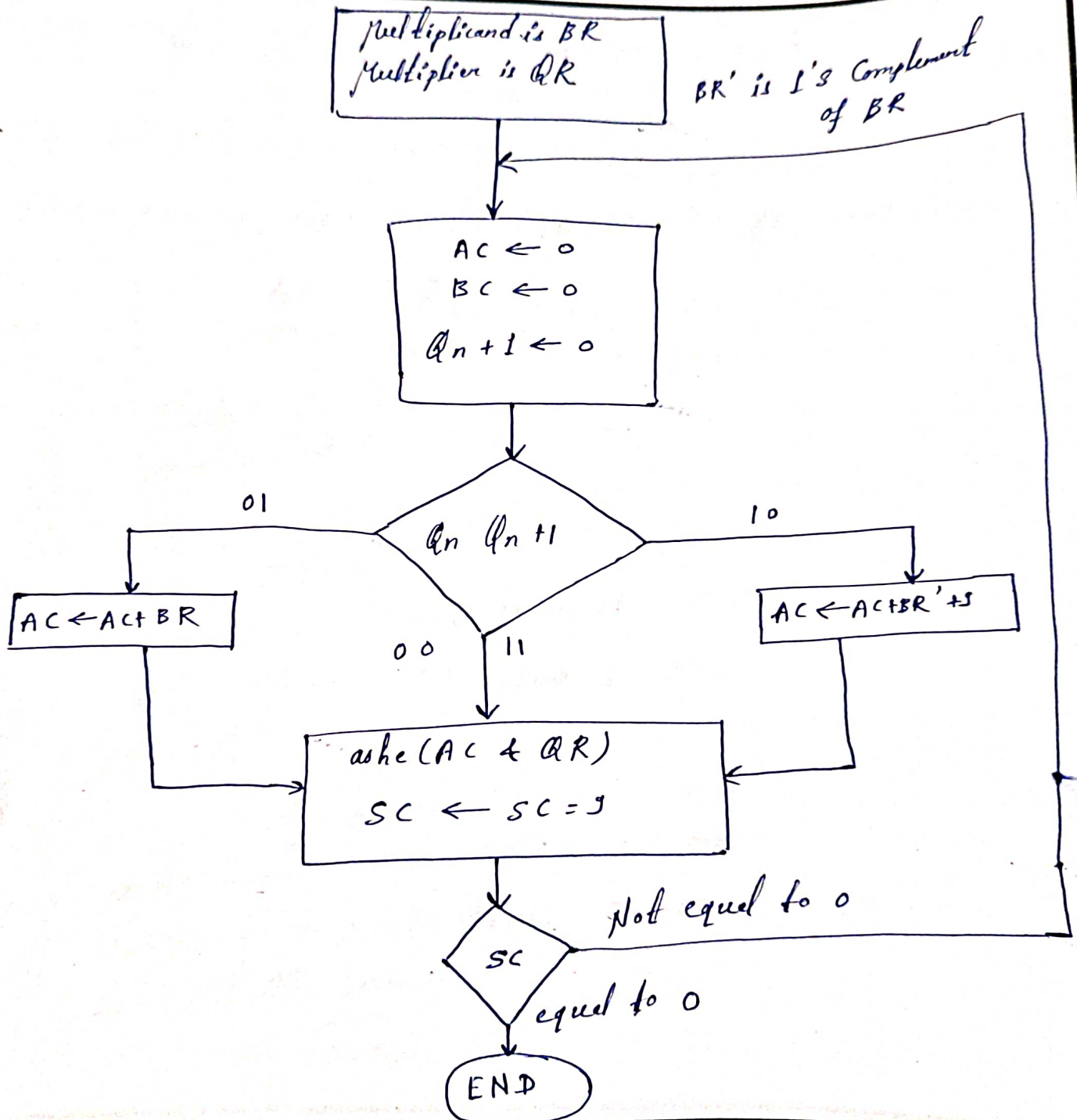
We name the register as A, B and Q, AC, BR and QR respectively. Q_n designates the least significant bit of multiplier in the register QR. An extra flip-flop Q_{n+1} is appended to QR to facilitate a double inspection of the multiplier. The flowchart for the booth algorithm is shown below.



Topic:

Objective:

Outcomes:





Topic:

Objective:

Outcomes:

Q.6 Explain the process of the multiplication using Booth method. solve (i) ~~(+4) * (+3)~~ ~~(-4) * (-3)~~ ~~(-4) * (+3)~~

(ii) $(+15) * (-13)$

→ (ii) $(+15) * (-13) = -195 = (1100111101)_2$'s Comp.

BR = 011111 (+15); $\overline{BR} + 1 = 10001 (-15)$; AR = 10011 (-13)

$Q_n Q_{n+1}$	AC	AR	Q_{n+1}	SC
	00000	10011	0	101

10 Initial
subtract BR

10001
10001

11 ashr — 11000 11001 1 100

01 ashr — 11100 01100 1 011

01 add BR 01111

01011

00 ashr — 00101 10110 0 010

00 ashr — 00010 11011 0 001

10 subtract BR 10001

10011



Sushila Devi Bansal College of Technology

Umariya, A.B. Road, Indore

Page No.

ashu

$$\begin{array}{r} 110011101 \\ \hline -195 \end{array}$$

1 000

(i) $(-4) \times 3$

$$(-4) = (1100)_2 = M, -M = 0100$$

$$3 = (0011)_2 = Q$$

AC	Q	Q-1
0000	0011	0

operation
 $AC = AC + (-M)$

$$\begin{array}{r} 0000 \\ 0100 \\ \hline 0100 \end{array}$$

0100	0011	0
0010	0001	1
0001	0000	1

ASR { Arithmetic shift right }

$$\begin{array}{r} AC = AC + M \\ = 0001 \\ + 0011 \\ \hline 0100 \end{array}$$

0100	0000	1
0010	0000	0
0001	0000	0

ASR

-12

Summary :



Topic:

Objective:

Outcomes:

Q.7. perform the arithmetic operations below with binary numbers and with negative numbers in signed 2's complement representation. Use seven bits to accommodate each number together with its sign. Also, check the overflow in each case.

(i) $(+35) + (+40)$

(ii) $(-35) + (-40)$

(iii) $(-35) + (+40)$

→ Binary of 35 : (0100011)

→ Binary of 40 : (0101000)

The rules for detecting overflow in a two's complement sum are simple:

- (1) If the sum of two positive numbers yields a negative result, the sum has overflowed.
- (2) If the sum of two negative numbers yields a positive result, the sum has overflowed.
- (3) Otherwise, the sum has not overflowed.

(i) $(+35) + (+40)$ 7 bit representation.

binary of 35 - $\boxed{0}100011$



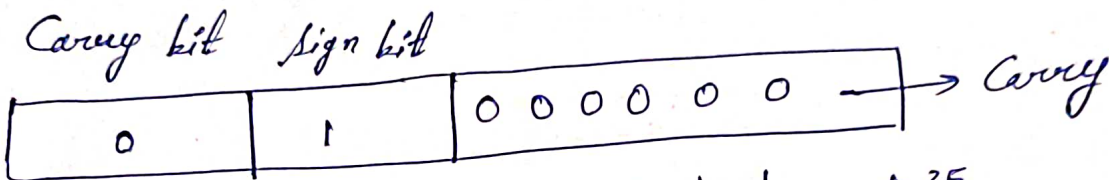
Topic:

Objective:

Outcomes:

Binary of 40 - 0101000

first bit represent (+ or -) sign of the no.



0 100011 +35

0 101000 +40

1 001011

$\left\{ \begin{array}{l} \text{Carry bit} \\ 0 \end{array} \right.$
 $\left\{ \begin{array}{l} \text{sign bit} \\ 1 \end{array} \right.$
 $\left. \right\}$ Overflow Condition

Qii) $(-35) + (-40)$

binary of 35 = 0100011

1's Complement of 35 = 1011100

2's Complement of 35 or -35 = 1011100
+1

1011101 (-35)



Topic:

Objective:

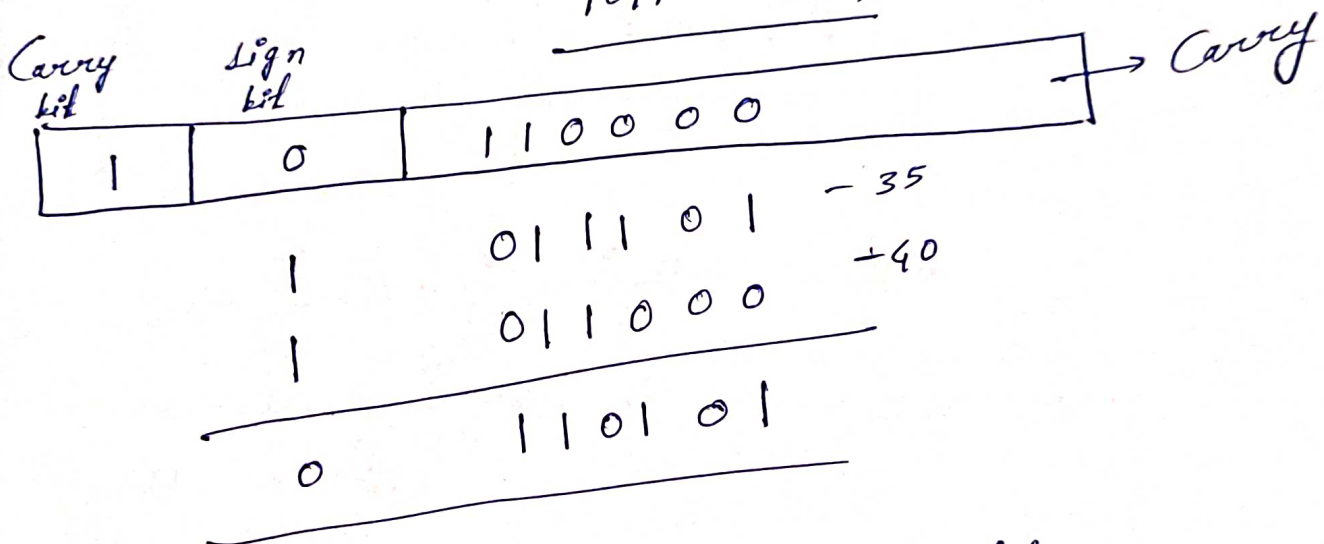
Outcomes:

Binary of 40 = 0101000

1's Complement of 40 = 1010111

2's Complement of 40 = 1010111
+1

1011000 (-40)



Carry bit 1 } underflow Condition
Sign bit 0 }

(iii) $(-35) - (+40) = (-35) + (-40)$

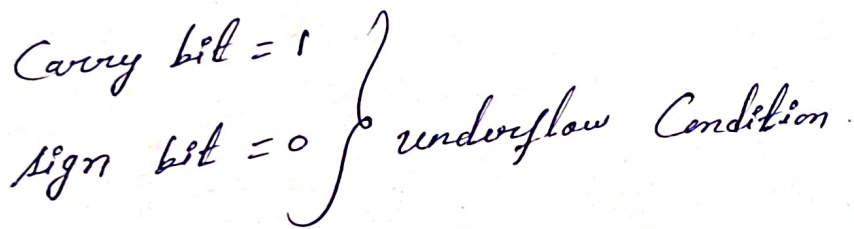
2's Complement of 35 or -35 = 1011101

2's Complement of 40 or -40 = 1011000



Page No. _____

Outcomes:


$$\begin{array}{r} 101101 - 35 \\ 0101000 + 40 \\ \hline 10000101 \end{array}$$